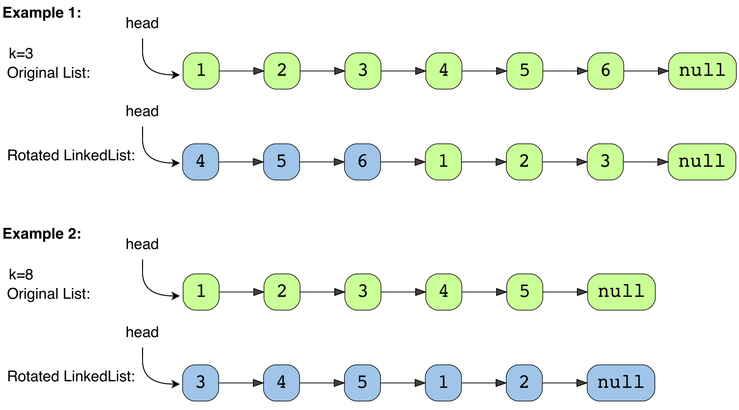
Solution: Problem Challenge 2: Rotate a LinkedList

Problem Statement

Given the head of a Singly LinkedList and a number ‘k’, rotate the LinkedList to the right by ‘k’ nodes.



Solution

Another way of defining the rotation is to take the sub-list of ‘k’ ending nodes of the LinkedList and connect them to the beginning. Other than that we have to do three more things:

1. Connect the last node of the LinkedList to the head, because the list will have a different tail after the rotation.
2. The new head of the LinkedList will be the node at the beginning of the sublist.
3. The node right before the start of sub-list will be the new tail of the rotated LinkedList.

Code

Here is what our algorithm will look like:

```java
import java.util.*;

/*class ListNode {
  int val = 0;
  ListNode next;

  ListNode(int Val) {
    this.val = Val;
  }
}*/

class Solution {

  public ListNode rotate(ListNode head, int rotations) {
    if (head == null || head.next == null || rotations <= 0)
      return head;

    // find the length and the last node of the list
    ListNode lastNode = head;
    int listLength = 1;
    while (lastNode.next != null) {
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The time complexity of our algorithm will be O(N) where ‘N’ is the total number of nodes in the LinkedList.

Space Complexity

We only used constant space, therefore, the space complexity of our algorithm is O(1).